Day 4 – ESP32 Telegram IoT Notification Preparation

---

1. Objective

The objective of **Phase 2 Day 4** was to move VisionGuard from local ESP32 alerting toward **remote IoT notification alerting**.

In earlier days of Phase 2:

```
Day 1 → ESP32 hardware setup completed
Day 2 → ESP32 WiFi foundation completed
Day 3 → Python AI detection integrated with ESP32 using serial
communication
```

On **Day 4**, the main goal was to test whether ESP32 could send a remote alert message using **Telegram Bot API**.

The focus was not yet full AI-triggered Telegram automation.
The goal was to prepare and verify the Telegram notification foundation.

---

2. Day 4 Main Goal

The target flow for Day 4 was:

```
ESP32
→ Connect to WiFi
→ Send Telegram test message
→ Receive message on Telegram app
→ Confirm success using LED and buzzer
→ Test failure cases
→ Protect secret credentials before GitHub upload
```

This step prepares the system for the next stage where the Python AI detection system will trigger ESP32, and ESP32 will send remote Telegram alerts.

---

3. Work Completed

The following tasks were completed on Phase 2 Day 4:

```
✅ Day 4 project folder created
✅ Telegram bot created using BotFather
✅ Bot token generated and saved privately
✅ Telegram Chat ID collected using getUpdates API
✅ UniversalTelegramBot library installed
```

```
✅ ArduinoJson library installed
✅ ESP32 Telegram test message sent successfully
✅ Telegram message received on mobile app
✅ LED and buzzer confirmation added
✅ WiFi failure case tested
✅ Wrong bot token failure case tested
✅ Secret credentials protected before GitHub upload
```

---

4. Hardware Components Used

Component	Purpose
ESP32 WiFi + Bluetooth Board	Main IoT microcontroller
External LED	Visual status indication
Active Buzzer	Audio confirmation and failure warning
220Ω Resistor	LED current protection
Breadboard	Temporary hardware setup
Jumper Wires	Circuit connections
USB Cable	ESP32 programming and power
Laptop	Arduino IDE programming
Mobile Phone	Telegram message receiving

---

5. Software and Libraries Used

Software / Library	Purpose
Arduino IDE	Programming ESP32
ESP32 Board Support Package	ESP32 development support
WiFi.h	ESP32 WiFi connection
WiFiClientSecure.h	Secure HTTPS communication
UniversalTelegramBot.h	Telegram Bot API communication
ArduinoJson	JSON handling required by Telegram library
Telegram App	Receiving bot alert messages
BotFather	Creating Telegram bot

---

6. Final Working Configuration

The working pin configuration from previous Phase 2 tasks was continued.

```
Board      : ESP32 Dev Module
Port       : COM4
Baud Rate  : 9600
LED Pin    : GPIO 23
Buzzer Pin : GPIO 22
WiFi       : 2.4 GHz Network
```

The LED and buzzer were used for status indication:

Condition	LED Output	Buzzer Output
Connecting to WiFi	LED blinks	OFF
Telegram message sent successfully	LED blinks once	Buzzer beeps once
WiFi or Telegram failure	LED stops / does not confirm	Buzzer beeps three times

---

7. Task 1 – Day 4 Folder Setup

7.1 Purpose

A separate folder was created for Day 4 to keep Telegram IoT notification files organized.

7.2 Suggested Folder Structure

```
Phase-2-ESP32-IoT-Alerting/
│
├── Day-4-Telegram-IoT-Notification/
│   ├── esp32_telegram_test/
│   │   └── esp32_telegram_test.ino
│   │
│   ├── esp32_wifi_telegram_alert/
│   │   └── esp32_wifi_telegram_alert.ino
│   │
│   └── notes/
│       └── phase2_day4_notes.md
```

7.3 Outcome

The Day 4 structure was created successfully.
This helped separate Telegram notification work from earlier WiFi and serial integration tasks.

---

8. Task 2 – Telegram Bot Creation

8.1 Purpose

A Telegram bot was needed so that ESP32 could send alert messages to the user through Telegram.

8.2 Method

The bot was created using **BotFather** on Telegram.

Commands used:

```
/start
/newbot
```

BotFather asked for:

```
Bot name
Bot username
```

After successful creation, BotFather generated a **Bot Token**.

8.3 Important Security Point

The bot token was treated as private information.

The real token was not shared publicly and was not uploaded to GitHub.

8.4 Outcome

Telegram bot was created successfully and the bot token was generated.

---

9. Task 3 – Telegram Chat ID Collection

9.1 Purpose

To send a message to a specific Telegram account, ESP32 needs the receiver's **Chat ID**.

9.2 Method

First, a message was sent to the created bot:

```
/start
hello
```

Then the Telegram Bot API `getUpdates` method was used in the browser:

```
https://api.telegram.org/botYOUR_BOT_TOKEN/getUpdates
```

Initially, the response showed:


```
{"ok":true,"result":[]}
```


9.3 Challenge Faced

The result was empty because the bot had not received any message yet.

9.4 Solution

The created bot was opened directly, and a message was sent to it.
After refreshing the `getUpdates` URL, the Chat ID appeared inside the response.

Example structure:


```
"chat":{"id":123456789}
```


9.5 Outcome

The Chat ID was successfully collected and saved privately.

---

10. Task 4 – Required Library Installation

10.1 Purpose

ESP32 needed Telegram-related libraries to communicate with the Telegram Bot API.

10.2 Libraries Installed

In Arduino IDE:

```
Sketch → Include Library → Manage Libraries
```

The following libraries were installed:

```
UniversalTelegramBot
ArduinoJson
```

10.3 Why These Libraries Are Needed

Library	Purpose
UniversalTelegramBot	Sends and receives Telegram bot messages
ArduinoJson	Handles JSON data used by Telegram API
WiFiClientSecure	Enables HTTPS connection from ESP32

10.4 Outcome

Required Telegram libraries were installed successfully.

---

11. Task 5 – ESP32 Telegram Test Message

11.1 Purpose

The goal of this task was to verify that ESP32 could connect to WiFi and send a message to Telegram.

11.2 Code Used

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>

// WiFi details
const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

// Telegram bot details
#define BOT_TOKEN "YOUR_BOT_TOKEN"
#define CHAT_ID "YOUR_CHAT_ID"

WiFiClientSecure client;
UniversalTelegramBot bot(BOT_TOKEN, client);

void setup() {
  Serial.begin(9600);
  delay(1000);

  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi Connected Successfully!");
  Serial.print("ESP32 IP Address: ");
  Serial.println(WiFi.localIP());

  // Required for HTTPS connection
  client.setInsecure();

  Serial.println("Sending Telegram message...");

  bool messageSent = bot.sendMessage(
    CHAT_ID,
    "VisionGuard ESP32 Telegram test successful.",
    ""
  );

  if (messageSent) {
    Serial.println("Telegram message sent successfully.");
  } else {
    Serial.println("Telegram message failed.");
  }
```

```
}

void loop() {
}
```

11.3 Explanation of Important Code Sections

WiFi Library

```
#include <WiFi.h>
```

This allows ESP32 to connect to a WiFi network.

Secure Client Library

```
#include <WiFiClientSecure.h>
```

Telegram API works over HTTPS, so a secure client is required.

Telegram Library

```
#include <UniversalTelegramBot.h>
```

This library simplifies sending Telegram messages from ESP32.

Telegram Bot Object

```
WiFiClientSecure client;
UniversalTelegramBot bot(BOT_TOKEN, client);
```

This creates a secure client and connects it with the Telegram bot token.

HTTPS Certificate Handling

```
client.setInsecure();
```

This allows ESP32 to make HTTPS requests without manually adding Telegram's SSL certificate.

For a student prototype, this is acceptable.
For production-level systems, certificate validation should be handled more securely.

Sending Telegram Message

```
bool messageSent = bot.sendMessage(
  CHAT_ID,
  "VisionGuard ESP32 Telegram test successful.",
  ""
);
```

This sends a test message to the Telegram chat ID.

11.4 Expected Output

Serial Monitor:

```
WiFi Connected Successfully!
ESP32 IP Address: 192.168.x.x
Sending Telegram message...
Telegram message sent successfully.
```

Telegram app:

```
VisionGuard ESP32 Telegram test successful.
```

11.5 Outcome

The Telegram message was successfully received from the ESP32.

This confirmed:

```
ESP32 WiFi connection working
Telegram Bot API working
Bot token and Chat ID correct
ESP32 can send remote IoT notification
```

This was the major milestone of Day 4.

---

12. Task 6 – LED and Buzzer Confirmation

12.1 Purpose

After confirming Telegram message delivery, LED and buzzer confirmation were added.

This gave the system both:

```
Remote confirmation → Telegram message
Local confirmation  → LED and buzzer
```

12.2 Code Used

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>

#define LED_PIN 23
#define BUZZER_PIN 22

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
```

```
#define BOT_TOKEN "YOUR_BOT_TOKEN"
#define CHAT_ID "YOUR_CHAT_ID"

WiFiClientSecure client;
UniversalTelegramBot bot(BOT_TOKEN, client);

void successAlert() {
  digitalWrite(LED_PIN, HIGH);
  digitalWrite(BUZZER_PIN, HIGH);
  delay(300);
  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
}

void failureAlert() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(BUZZER_PIN, HIGH);
    delay(200);
    digitalWrite(BUZZER_PIN, LOW);
    delay(200);
  }
}

void setup() {
  Serial.begin(9600);
  delay(1000);

  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);

  int attempt = 0;

  while (WiFi.status() != WL_CONNECTED && attempt < 20) {
    digitalWrite(LED_PIN, HIGH);
    delay(200);
    digitalWrite(LED_PIN, LOW);
    delay(300);

    Serial.print(".");
    attempt++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("");
    Serial.println("WiFi Connected Successfully!");
    Serial.print("ESP32 IP Address: ");
    Serial.println(WiFi.localIP());

    client.setInsecure();

    Serial.println("Sending Telegram message...");

    bool messageSent = bot.sendMessage(
      CHAT_ID,
      "VisionGuard ESP32 Telegram test successful.",
```

```
      ""
    );

    if (messageSent) {
      Serial.println("Telegram message sent successfully.");
      successAlert();
    } else {
      Serial.println("Telegram message failed.");
      failureAlert();
    }
  } else {
    Serial.println("");
    Serial.println("WiFi Connection Failed!");
    failureAlert();
  }
}

void loop() {
}
```

12.3 Explanation of LED and Buzzer Logic

During WiFi Connection

```
while (WiFi.status() != WL_CONNECTED && attempt < 20) {
  digitalWrite(LED_PIN, HIGH);
  delay(200);
  digitalWrite(LED_PIN, LOW);
  delay(300);

  Serial.print(".");
  attempt++;
}
```

The LED blinks while ESP32 is trying to connect to WiFi.

If the LED blinks three times, it means ESP32 needed around three connection attempts before WiFi connected.

Success Alert Function

```
void successAlert() {
  digitalWrite(LED_PIN, HIGH);
  digitalWrite(BUZZER_PIN, HIGH);
  delay(300);
  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
}
```

This function runs only when the Telegram message is sent successfully.

Output:

```
LED blinks once
Buzzer beeps once
```

Failure Alert Function

```
void failureAlert() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(BUZZER_PIN, HIGH);
    delay(200);
    digitalWrite(BUZZER_PIN, LOW);
    delay(200);
  }
}
```

This function runs when WiFi connection fails or Telegram message sending fails.

Output:

```
Buzzer beeps three times
```

12.4 Outcome

The LED and buzzer confirmation worked successfully.

Observed behavior:

```
LED blinked during WiFi connection attempts
Telegram message was received
LED blinked once at the end
Buzzer beeped once at the end
```

This confirmed that the local and remote notification indicators were working together.

---

13. Task 7 – Failure Case Testing

13.1 Purpose

Failure testing was performed to check how the system behaves when connection or credentials are incorrect.

IoT systems should not only work in success conditions.
They should also give clear output during failure conditions.

13.2 Failure Case 1 – Wrong WiFi Password

Test

The WiFi password was temporarily changed to an incorrect value.

```
const char* password = "WRONG_PASSWORD";
```

Expected Result

```
WiFi Connection Failed!
Buzzer beeps three times
Telegram message is not sent
```

Outcome

The ESP32 failed to connect to WiFi as expected.
The buzzer gave warning beeps, confirming failure handling.

13.3 Failure Case 2 – Wrong Bot Token

Test

The bot token was temporarily changed to an incorrect value.

```
#define BOT_TOKEN "WRONG_BOT_TOKEN"
```

Expected Result

```
WiFi Connected Successfully!
Sending Telegram message...
Telegram message failed.
Buzzer beeps three times
```

Outcome

WiFi connected successfully, but Telegram message sending failed.
The failure was detected and the buzzer warning was triggered.

13.4 Final Restoration

After failure testing, correct credentials were restored.

The ESP32 again sent the Telegram message successfully.

---

14. Task 8 – Secret Credential Protection

14.1 Purpose

The project uses sensitive credentials:

```
WiFi name
WiFi password
Telegram bot token
Telegram chat ID
```

These should not be uploaded publicly to GitHub.

14.2 Action Taken

Real values were removed from GitHub-ready code and replaced with placeholders.

```
const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

#define BOT_TOKEN "YOUR_BOT_TOKEN"
#define CHAT_ID "YOUR_CHAT_ID"
```

14.3 Security Note for README

A security note should be added:

```
## Security Note

This project uses WiFi credentials, Telegram Bot Token, and Telegram Chat
ID for IoT notification testing.

For security reasons, real credentials are not uploaded to GitHub.
Before running the code, replace the placeholder values with your own
private credentials.
```

14.4 Outcome

The code was made GitHub-safe by removing real private credentials.

---

15. Challenges Faced and Outcomes

Challenge	Cause	Solution	Outcome
Telegram bot username already taken	Bot usernames must be unique	Used a unique project-related bot username	Bot created successfully
`getUpdates` returned empty result	Bot had not received any message yet	Sent `/start` and `hello` to the bot, then refreshed API URL	Chat ID received successfully
Telegram required secure HTTPS communication	Telegram API uses HTTPS	Used `WiFiClientSecure` and `client.setInsecure()`	ESP32 could communicate with Telegram
Need to confirm message delivery locally	Telegram message alone does not show local hardware status	Added LED and buzzer success indication	LED blinked once and buzzer beeped once after success
Need to handle failure cases	Wrong WiFi or wrong bot token can break notification	Added `failureAlert()` and tested wrong credentials	Buzzer warning worked during failure
Sensitive credentials risk	GitHub upload could expose password/token	Replaced real credentials with placeholders	Code became safe for public upload

---

16. Final Output Summary

At the end of Phase 2 Day 4, the following outputs were achieved:

```
✅ Telegram bot created
✅ Bot token generated
✅ Chat ID collected
✅ Telegram libraries installed
✅ ESP32 connected to WiFi
✅ ESP32 sent Telegram test message
✅ Telegram message received successfully
✅ LED blinked during WiFi connection
✅ LED and buzzer confirmed success
✅ Wrong WiFi failure tested
✅ Wrong bot token failure tested
✅ Credentials protected for GitHub
```